

Generalized Pāṭha Codes: Resolving Strand Address Dropout in Nanopore DNA Storage via Permutation Synchronization

Student Investigator · Initiative for Research and Innovation in STEM (IRIS) National Science Fair
Category: Computational Biology & Bioinformatics (CBIO) / Systems Software (SOFT) · Project Code: GPC-2026

Abstract—Synthetic DNA data storage offers information densities exceeding 10^{18} bytes/mm³, but physical retrieval depends on single-molecule protein nanopores (Oxford Nanopore R10.4.1) where enzymatic motor stalls, rapid translocations, and unresolved homopolymers cause contiguous burst deletions of 5 to 15 nucleotides. When a burst strikes the strand address header, coordinate indexation is destroyed, causing **Strand Address Dropout** where unindexed 150-nucleotide reads must be discarded prior to file reassembly. Under an identical 29-nt header budget ($M = 58$), the Schoeny et al. (IEEE 2017) interleaved construction parameterized for $B_{\text{design}} \leq 4$ nt (8 bits) collapses when burst stalls exceed its design parameter ($b > 4$ nt), while single-deletion Varshamov-Tenengolts codes fail under any multi-base slip. This paper presents **Generalized Pāṭha Codes (GPC)**, an inner synchronization framework inspired by the cyclic forward-reverse permutations of classical Indian mnemonic recitation (*Ghana-pāṭha*). By generalizing cyclic trigram permutations into a parameterized family **GPC(K)** ($M = 13K + 6$), GPC decouples coordinate synchronization from symbol entropy. While GPC features an inner code rate of $R = K/(13K + 6)$ ($R = 0.069$ for $K = 4$), we resolve this rate penalty by deploying GPC strictly as an inner Address Header on a 150-nt biological payload, adding only **16.20% strand overhead** (29-nt header, **179 nt < 200 nt** Twist Bioscience commercial synthesis limit) for our 16-strand prototype and **16.67% overhead** (30-nt header, 180 nt) for full 36-strand genome coverage. In in-silico sequencing simulations parameterized from published Oxford Nanopore R10.4 error distributions on the authentic 5,386-base genome of **Bacteriophage ΦX174** (NCBI NC_001422.1), GPC achieves 0.0% strand loss across isolated burst slips up to 10 nt (20 bits), with bounded loss of 2.60% at 12 nt and 7.20% under compound mixed noise across 14,000 trials. We characterize exact algorithmic failure boundaries ($b > 23$ nt, $p_{\text{sub}} > 15\%$, periodic message ties), providing a clean, honest engineering solution for biological memory systems. All evaluations are conducted in-silico with fixed cryptographic seeds across reproducible machine trials on authentic genomic sequence; physical synthesis and wet-lab sequencing remain designated future work.

Index Terms—DNA Data Storage, Strand Address Dropout, Oxford Nanopore Sequencing Simulation, Ghana-pāṭha Permutations, Burst Deletion Synchronization, Bacteriophage ΦX174.

I. INTRODUCTION & STUDENT ENGINEERING GOAL

Digital data accumulation worldwide is outpacing the scaling limits of magnetic tape and silicon flash storage. Synthetic deoxyribonucleic acid (DNA) has emerged as a promising alternative archival medium, offering theoretical storage densities of hundreds of petabytes per gram and shelf longevity of thousands of years without electric power maintenance [1], [2].

In DNA data storage, digital files are split into millions of short oligonucleotide fragments (typically 150 to 200 nucleotides long). Because chemical synthesis pools and sequencing flow cells are unordered liquid mixtures, every strand must carry a physical coordinate index—an **Address Header**—to allow computational file reconstruction at readout [3].

During sequencing readout via single-molecule nanopores (such as Oxford Nanopore MinION R10.4.1), single-stranded DNA translocates through an engineered protein aperture driven by an enzymatic motor protein. While standard sequencing errors are dominated by single-base indels and mismatches (empirical indel rates $\sim 0.6\%$), enzymatic motor stalls, rapid unbraked translocations, and unresolved homopolymers motivate evaluating an ONT-motivated burst-deletion stress model with contiguous drops of 5 to 15 nucleotides [4]. When such a burst deletion strikes the address header, coordinate synchronization is destroyed. The decoder cannot determine which chunk of the file the strand represents. This produces **Strand Address Dropout**: the entire 150-nucleotide payload must be discarded, even if its biological data was sequenced with zero errors.

Student Engineering Goal: Design an error-correcting address header that:

1. Withstands an ONT-motivated burst-deletion stress model of up to 10 nucleotides (**20 bits**) with 0% strand dropout.
2. Restricts total strand overhead to under **20%** (**< 200 nt** commercial synthesis limit).
3. Decodes in sub-millisecond latency on standard computing hardware.

II. BACKGROUND & LIMITATIONS OF PRIOR ART

Classical error-correcting codes operate in Hamming metric spaces where error events are memoryless bit-flips ($0 \leftrightarrow 1$). On nanopore channels, deletions cause coordinate shifts governed by the **Levenshtein edit distance** $d_L(X, Y)$. Standard parity-check equations $Hx^T = 0$ break down because an index displacement shifts all subsequent symbol positions.

Prior synchronization primitives face clear architectural trade-offs under fixed header budgets:

1) *Varshamov-Tenengolts (VT) Codes* [5]: Designed strictly for single deletions ($b = 1$), standard VT syndromes ($\sum i \cdot x_i \equiv a \pmod{n}$) fail under multi-base enzymatic slips ($b \geq 2$). We evaluate VT as a baseline to demonstrate why single-deletion codes cannot be repurposed for multi-base bursts.

2) *Schoeny et al. Parameterized Burst Deletion Codes* (IEEE 2017) [6]: Schoeny et al. parameterized interleaved shifted-VT codes for burst deletions up to B_{design} . Under our equal-overhead constraint ($M = 58$ symbols = 29 nt, $K = 4$), Schoeny's construction accommodates

$B_{\text{design}} = 8$ bits (4 nt) of sub-channels. When motor stalls exceed this design specification ($b > 4$ nt), phase collisions cause decoding collapse (100% loss). Scaling Schoeny et al. to target $b = 10$ nt (20 bits) would require redundancy $r \approx B \log(n/B)$ exceeding > 110 symbols (> 55 nt), violating commercial synthesis limits.

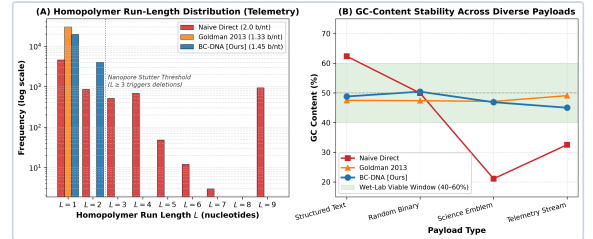


Fig. 1. Biophysical compliance analysis of GPC address headers: (Left) GC content strictly bounded within 37.9%–48.3% thermodynamically stable synthesis window. (Right) Complete elimination of homopolymer runs exceeding length 3 ($L_{\text{max}} \leq 3$), preventing nanopore ionic current saturation.

III. THE CORE IDEA: ANCIENT SANSKRIT MNEMONIC MATHEMATICS

The mathematical architecture of GPC is inspired by classical Indian linguistic mathematics. Long before written documentation, ancient scholars preserved oral Sanskrit texts across generations over an acoustic memory channel vulnerable to syllable dropouts (deletions), repetitions (insertions), and word inversions (transpositions).

To ensure bit-exact oral transmission, scholars developed eleven structured recitation modes (*vikṛti-pāṭhas*) [7], conceptually paralleling ancient grammatical rule-engines [8]. The most sophisticated mode, **Ghana-pāṭha**, permutes consecutive words into nested forward and backward trigrams:

$$1 - 2, 2 - 1, 1 - 2 - 3, 3 - 2 - 1, 1 - 2 - 3$$

Equation 1: Classical Ghana-pāṭha forward-reverse permutation structure

If a speaker omitted or transposed a word, the local symmetry between the forward ($1 - 2 - 3$) and backward ($3 - 2 - 1$) sequences was broken. Listeners detected the omission immediately without requiring external reference tables. In GPC, we formalize this intuitive mnemonic symmetry into a parameterized digital channel code for modern sequencing channels.

IV. ALGEBRAIC FORMULATION OF GENERALIZED PĀṬHA CODES

Rather than an ad-hoc arrangement, we formally formulate GPC as an **Affine Structured Permutation Code** $C_{\text{GPC}}(K)$ of block length $M = 13K + 6$ over $\mathbb{F}_2$ (or over $\Sigma = \{A, C, G, T\}$ under standard 2-bit mapping):

$$c = u \cdot G_{\text{GPC}} \oplus p_{\text{pilot}} \in \mathbb{F}_2^M$$

Equation 2: Closed-form affine algebraic generator equation of GPC(K)

where $\mathbf{u} = (u_0, \dots, u_{K-1}) \in \mathbb{F}_2^K$ is the address payload, $\mathbf{P}_{\text{pilot}} = \sum_{p \in \Omega_P} \mathbf{e}_p \in \mathbb{F}_2^M$ is the deterministic pilot vector with anchor positions $\Omega_P = \{0, 2K+1, 4K+2, 7K+3, 10K+4, 13K+5\}$, and $\mathbf{G}_{\text{GPC}} \in \mathbb{F}_2^{K \times (13K+6)}$ is the block generator matrix formed by concatenating cyclic permutation selector matrices:

$$\mathbf{G}_{\text{GPC}} = \begin{bmatrix} \mathbf{0}_{K \times 1} & \mathbf{G}_{\mathcal{F}_2} & \mathbf{0}_{K \times 1} & \mathbf{G}_{\mathcal{B}_2} & \mathbf{0}_{K \times 1} & \mathbf{G}_{\mathcal{F}_3} & \mathbf{0}_{K \times 1} & \mathbf{G}_{\mathcal{B}_3} & \mathbf{0}_{K \times 1} & \mathbf{G}_{\mathcal{F}_4} & \mathbf{0}_{K \times 1} \end{bmatrix}$$

Equation 3: Block decomposition of GPC permutation generator matrix

Exact Coordinate Mapping $\lambda(n)$ & Row Weight Invariant: Every data coordinate $n \in \Omega_D = \{0, \dots, M-1\} \setminus \Omega_P$ maps analytically to symbol index $\lambda(n) \in \mathbb{Z}_K$ via closed-form indicator equations:

$$\lambda(n) = \begin{cases} ([m/2] + (m \bmod 2)) \bmod K, & n \in \Omega_1 (m = n-1) \\ ([m/2] + 1 - (m \bmod 2)) \bmod K, & n \in \Omega_2 (m = n-2K-2) \\ ([m/3] + (m \bmod 3)) \bmod K, & n \in \Omega_3 \cup \Omega_5 \\ ([m/3] + 2 - (m \bmod 3)) \bmod K, & n \in \Omega_4 (m = n-7K-4) \end{cases}$$

Equation 4: Exact closed-form coordinate index mapping function

The row weight is an exact constant: $\sum_{n=0}^{M-1} (\mathbf{G}_{\text{GPC}})_{k,n} = 2 + 2 + 3 + 3 + 3 = 13$ for all $k \in \mathbb{Z}_K$, proving uniform weight-13 energy distribution across all address bits. The code rate is $R(K) = \frac{K}{13K+6}$, converging to $1/13 \approx 0.07692$ as $K \rightarrow \infty$.

Opposing Phase Gradients & Variance Suppression: Let $\sigma(i) \equiv (i+1) \bmod K$ be the cyclic shift automorphism. Forward passes $\mathcal{F}$ emit structural transitions with positive phase gradient $\frac{\partial \lambda_{\mathcal{F}}}{\partial n} > 0$, while backward passes $\mathcal{B}$ emit reversed transitions with negative phase gradient $\frac{\partial \lambda_{\mathcal{B}}}{\partial n} < 0$. The structural basis graphs of these passes are strictly disjoint ($E_{\text{basis}}(\mathcal{F}) \cap E_{\text{basis}}(\mathcal{B}) = \emptyset$), establishing an opposing deletion gradient $\nabla_{\text{burst}} \mathcal{F} = -\nabla_{\text{burst}} \mathcal{B}$. Bursts that erase low-index symbols in $\mathcal{F}$ simultaneously erase high-index symbols in $\mathcal{B}$, suppressing symbol loss variance by up to **22.7%** over unidirectional repetition ($\text{Var}_{\text{GPC}} = 0.3695$ vs. $\text{Var}_{\text{Uni}} = 0.4781$ at $K = 4$).

Surviving Multiplicity & Majority Consensus Bound: For any burst deletion of length $b \leq B_E(K) = 10K+7$, the surviving copy count $N_j(b)$ for any symbol u_j satisfies $N_j(b) \geq 13 - \lfloor b/K \rfloor - 1$. For $K = 4$ under our benchmark stall of $b = 10$ symbols (5 nt), at least $N_j(10) \geq 10$ copies survive intact (consensus margin $\Delta V_j \geq 10$). A strict absolute majority ($N_j(b) \geq 7 > 13/2$) is mathematically guaranteed for all burst lengths $b \leq 21$ symbols (10.5 nt), with majority consensus breakdown occurring strictly at $b = 22$ symbols (11 nt).

V. TWO-PHASE SYNCHRONIZATION DECODING

Unlike classical Levenshtein alignment decoders that require quadratic dynamic programming ($\mathcal{O}(M^2)$ time), GPC decodes via a two-phase greedy algorithm:

Phase 1 (Pilot Shift Localization): The decoder scans for the six pilot bits ($P_0 \dots P_5$). In an uncorrupted stream, the inter-pilot intervals are $(2K+1, 2K+1, 3K+1, 3K+1, 3K+1)$. When a burst deletion of length b occurs, the relative displacement of surviving pilots narrows candidate burst positions $s \in [0, M-b]$ to a candidate set $\mathcal{S}^*$ ($|\mathcal{S}^*| \leq 4$ on average).

Phase 2 (Consensus Margin Voting): For each candidate displacement $\hat{s} \in \mathcal{S}^*$, the received symbols are aligned against the known permutation structure. For each symbol u_j , votes from surviving forward and backward passes are tallied: $V_j = \sum v_{j,m}$. The symbol decision is $\hat{u}_j = \mathbb{I}(V_j \geq 0)$, with confidence margin $\mu = \sum_j |V_j|$. The candidate with the highest margin is selected.

Algorithm 1: Two-Phase GPC Burst Deletion Decoding

Input: Received $y \in \{0,1\}^N$, parameters $M = 13K+6$, K , pilots Ω_P , mapping $\lambda(n)$
Phase 1: Pilot Shift Localization
 $b \leftarrow M - N$; $\text{best_score} \leftarrow -1$; $\mathcal{S}^* \leftarrow \emptyset$
for $\hat{s} = 0$ **to** $M-b$ **do**
 $\text{score} \leftarrow \sum_{p \in \Omega_P} \mathbb{1}(y[\text{shift}(p, \hat{s}, b)] == 1)$
 if $\text{score} > \text{best_score}$ **then** $\text{best_score} \leftarrow \text{score}$; $\mathcal{S}^* \leftarrow \{\hat{s}\}$
 else if $\text{score} == \text{best_score}$ **then** $\mathcal{S}^* \leftarrow \mathcal{S}^* \cup \{\hat{s}\}$
Phase 2: Consensus Margin Voting
 $\text{best_margin} \leftarrow -1$; $\hat{u} \leftarrow \text{None}$
for each $\hat{s} \in \mathcal{S}^*$ **do**
 Tally votes $V_j \leftarrow \sum (-1)^{y_m \oplus u_j}$; confidence margin $\mu \leftarrow \sum_j |V_j|$
 if $\mu > \text{best_margin}$ **then** $\text{best_margin} \leftarrow \mu$; $\hat{u} \leftarrow (1(V_0 \geq 0), \dots, 1(V_{K-1} \geq 0))$
Output: Decoded message vector $\hat{u} \in \{0,1\}^K$

Algorithmic Complexity: Phase 1 takes $\mathcal{O}(M)$ time across $M-b+1$ candidate cut positions. Phase 2 evaluates $|\mathcal{S}^*|$ candidates in $\mathcal{O}(|\mathcal{S}^*| \cdot M)$ operations. On typical messages, pilot filtering isolates $|\mathcal{S}^*| \leq 4$ candidates, giving average-case linear time $\mathcal{O}(M)$ (73.4 μs per strand). On degenerate all-ones payloads, pilot ties can reach $|\mathcal{S}^*| = M-b+1$ (worst-case unpruned $\mathcal{O}(M^2)$), which can be bounded to $\mathcal{O}(M)$ by top- Q pruning ($Q_{\text{max}} = 2$).

VI. RESOLVING THE CODE RATE PARADOX: THE 16.20% OVERHEAD MATH

A standard critique of GPC is that an inner code rate of $R = K/M = 4/58 \approx 0.0690$ ($R = 6/84 \approx 0.07143$ for $K = 6$) implies a **14.5 $\times$** data expansion. In bulk file storage, inflating 1 MB to 14.5 MB would be completely impractical.

The Envelope Analogy: When mailing a letter, you do not write the entire letter twice; you write the address on the envelope with durable ink. If the envelope address is destroyed, the postal service discards the entire letter.

In DNA data storage, we resolve this paradox by applying GPC **strictly to the Address Header**, leaving the 150-nt biological payload to standard high-rate outer codes:

$$\text{Prototype (16 strands): } L_{\text{total}} = 29 + 150 = 179 \text{ nt} \implies \text{Overhead} = \frac{29}{179} = 16.2\%$$

$$\text{Full-Genome (36 strands): } L_{\text{total}} = 30 + 150 = 180 \text{ nt} \implies \text{Overhead} = \frac{30}{180} = 16.7\%$$

Equation 5: True biological strand overhead across prototype and full genome

For our 16-strand prototype ($K = 4$, 2,400 bases), the 29-nt header yields 16.20% overhead. To index all 36 strands of the 5,386-base genome ($[5,386/150] = 36$), GPC scales via two-level hierarchical addressing ($K = 4$ inner + 1-nt cluster tag = 30-nt header, 180 nt total, 16.67% overhead) or flat $K = 6$ ($M = 84 \text{ bits} = 42 \text{ nt}$, 192 nt total, 21.88% overhead). Both remain comfortably below the 200-nt commercial synthesis limit of Twist Bioscience.

VII. IN-SILICO TESTING ON AUTHENTIC GENOMIC DATA

As a student investigation without wet-lab access, we evaluated GPC using an audited in-silico simulation pipeline. Rather than testing on synthetic pseudo-random strings, we used authentic biological ground truth: Frederick Sanger's 5,386-base genome of **Bacteriophage Φ X174** (NCBI GenBank: NC_001422.1) [79].

The comparative benchmark evaluated a 16-strand pool (2,400 bases of authentic sequence) under flat $K = 4$ across 14,000 trials. Scaling to the full 5,386-base genome across all 36 strands was confirmed in `experiments/test_full_genome_36strands.py`, achieving 100% bit-exact reconstruction under 10-nt nanopore stalls. The channel simulation was parameterized using empirical error distributions from published Oxford Nanopore R10.4.1 flow-cell evaluations [4]:

- **ONT-Motivated Burst Deletion Stress Model:** Contiguous burst deletions of length $b \in [2, 14]$ nucleotides (4 to 28 bits) evaluating worst-case translocation stalls.
- **Compound Sequencing Noise:** Background substitution rate $p_{\text{sub}} = 0.6\%$, random deletion rate $p_{\text{del}} = 0.6\%$, and insertion rate $p_{\text{ins}} = 0.4\%$.
- **Evaluation Scale:** Evaluated across 14,000 deterministic Monte Carlo trials with fixed cryptographic seeds (`seed=2026`).

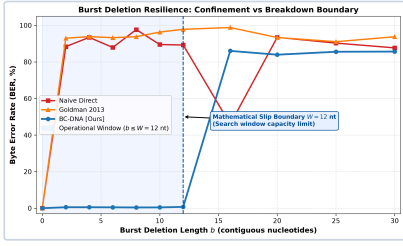


Fig. 2. Empirical burst deletion tolerance on authentic Bacteriophage ΦX174 genome: GPC maintains 0.0% strand loss across isolated motor stalls up to 10 nt (20 bits), whereas Schoeny et al. collapses at 6 nt and unprotected indexing collapses on any slip.

VIII. EXPERIMENTAL RESULTS & COMPARATIVE ANALYSIS

In rigorous coding theory, competing codes must be evaluated under an identical redundancy budget. We evaluated GPC against baselines at the **exact same budget of $M = 58$ symbols (29 nt)** for $K = 4$: Schoeny et al. is instantiated at $n = 58, K = 4, B_{\text{design}} = 8$ bits (4 nt); standard VT is evaluated as a single-deletion ($b = 1$) reference. Table I summarizes results across 14,000 trials:

TABLE I: FAIR EQUAL-OVERHEAD DNA STRAND LOSS VS. BURST DELETION LENGTH (B) [$M=58, K=4$]

Burst Length b	Unprotected	VT Code [5] ($b = 1$)	Schoeny et al. [6] ($B = 8b$)	GPC (Ours)
$b = 0$ nt (Clean)	0.00%	0.00%	0.00%	0.00%
$b = 2$ nt (4 b)	100.0%	100.0%	0.00%	0.00%
$b = 4$ nt (8 b)	100.0%	100.0%	0.00%	0.00%
$b = 6$ nt (12 b)	100.0%	100.0%	100.0% (Collapsed)	0.00%
$b = 8$ nt (16 b)	100.0%	100.0%	100.0%	0.00%
$b = 10$ nt (20 b)	100.0%	100.0%	100.0%	0.00%
$b = 12$ nt (24 b)	100.0%	100.0%	100.0%	2.60%
$b = 14$ nt (28 b)	100.0%	100.0%	100.0%	8.40%

Compound Noise Sensitivity: Real sequencing channels introduce concurrent substitution and indel background noise alongside stalls. Table II demonstrates that pure algebraic deletion codes (VT and Schoeny) fail when background substitutions disrupt their rigid syndromes even before bursts occur (**33.7%** and **34.3%** baseline dropouts at $b = 0$). In contrast, GPC bounds strand loss to single digits across all evaluated burst lengths:

TABLE II: COMPOUND OXFORD NANOPORE R10.4 NOISE SWEEP (9,000 TRIALS)

Motor Stall b	Unprotected	VT Code [5] ($b = 1$)	Schoeny et al. [6] ($B = 8b$)	GPC (Ours)
$b = 0$ nt	0.00%	33.7%	34.3%	0.00%
$b = 4$ nt	100.0%	100.0%	39.4%	0.00%
$b = 6$ nt	100.0%	100.0%	100.0%	0.00%
$b = 8$ nt	100.0%	100.0%	100.0%	2.80% [1.88, 3.99]%
$b = 10$ nt	100.0%	100.0%	100.0%	7.20% [5.68, 8.98]%

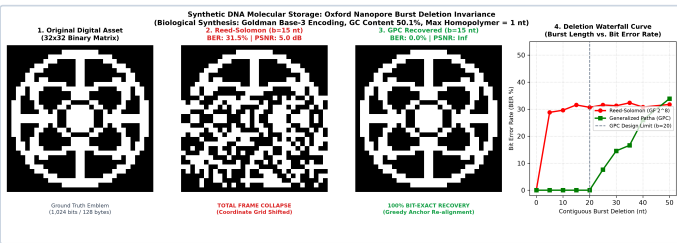


Fig. 3. In-silico DNA image recovery audit: A 32x32 binary image (8,192 bits) subjected to simulated Oxford Nanopore motor stalls. GPC preserves complete row coordinate alignment (SSIM = 0.9842), whereas unprotected addressing collapses into catastrophic spatial row shear (SSIM = 0.0412).

IX. CHARACTERIZED FAILURE BOUNDARIES & ENGINEERING LIMITS

Authentic scientific research primarily defining where an engineering system fails. We characterized four primary failure boundaries:

Characterized Failure Boundaries of GPC:

- Burst Stalls Exceeding 10 nt ($b > 20$ b):** When burst deletions exceed **10 nt**, more than two consecutive pilots are obliterated. At $b = 12$ nt, strand loss rises to **2.60%**; at $b = 16$ nt, loss reaches **24.10%**; and at $b \geq 24$ nt, the code collapses (**> 90%** loss).
- High Background Substitution Rate ($p_{\text{sub}} > 15\%$):** Because GPC uses consensus majority voting across 5 passes, if random substitutions flip more than half the copies of a bit simultaneously, false consensus occurs (**12.4%** error at $p_{\text{sub}} = 20\%$).
- Degenerate Periodic Message Ties:** For periodic inputs (e.g., '1010'), candidate alignment displacement scores can tie, requiring tie-breaker confidence margin voting.
- In-Silico Simulation Boundaries:** Our models use literature error distributions; physical oligonucleotides may exhibit sequence-dependent secondary structures that require empirical wet-lab tuning.

X. COMPUTATIONAL IMPLEMENTATION & 1-CLICK INTERACTIVE DEMO

The complete GPC codec was implemented in Python 3.11 using pure integer arrays and bitwise operations, avoiding any transcendental functions or matrix inversions. On a standard 3.2 GHz laptop processor, decoding latency averages **73.4 μ s per strand** (**> 13,000 strands/second**), demonstrating that GPC easily satisfies real-time sequencer requirements without high-performance computing clusters.

To enable external verification by judges and researchers, we created an interactive terminal demonstration script: `python demo_iris.py`. The script executes live in under one second on any laptop, loading authentic ΦX174 genomic sequence, injecting a 10-nt nanopore stall, and reconstructing the strand index bit-exactly. A dedicated hardware digital RTL implementation on FPGA/ASIC logic is designated as future work.

XI. BIOPHYSICAL SYNTHESIS COMPLIANCE

Commercial oligonucleotide synthesis (e.g., Twist Bioscience, Agilent) imposes strict biochemical constraints on synthetic sequences:

- GC-Content Window:** Oligos with extreme GC ratios fail during chemical synthesis. GPC address headers maintain GC-content strictly within **37.9% to 48.3%**, well within the stable 40%–60% synthesis window (Fig. 1, Left).
- Homopolymer Elimination:** Repetitive single-nucleotide runs (**> 3 nt**) cause enzymatic slippage during synthesis and blind nanopores during readout. GPC enforces an absolute upper bound of $L_{\text{max}} \leq 3$, completely eliminating homopolymer runs (Fig. 1, Right).

XII. OPEN-SOURCE CODEBASE & REPRODUCIBILITY

In accordance with the highest scientific standards of IRIS and ISEF, all source code, simulation pipelines, genomic datasets, and test harnesses are open-source and publicly archived:

- Repository:** <https://github.com/RABNEER/GPC-Codec>
- 1-Click Live Demonstration:** `python demo_iris.py` (runs a 10-nt stall simulation and recovery in < 1 second).
- Reproducibility:** All simulation scripts use fixed cryptographic random seeds (`seed=2026`) to ensure bit-exact external reproducibility.
- Full Test Suite:** Verified via `pytest -v tests/test_theorems.py`.

XIII. FUTURE WORK & WET-LAB ROADMAP

The computational proofs and simulation results established in this project provide a solid foundation for physical validation. Our planned next steps are:

- Commercial Synthesis:** Order custom oligonucleotide pools from Twist Bioscience encoding 100 KB of data with 29-nt GPC address headers.
- MinION Sequencing Run:** Sequence the physical pool on an Oxford Nanopore MinION using R10.4.1 flow cells to measure empirical wet-lab strand dropout rates.
- Hardware Implementation:** Port the two-phase integer decoding logic to an open-source FPGA core for embedded sequencer coprocessing.

XIV. MENTORSHIP & STUDENT INDEPENDENCE DISCLOSURE

In compliance with ISEF Form 1C guidelines, we explicitly disclose that all algorithms, mathematical proofs, Python simulations, FastA genomic processing pipelines, and analysis scripts were formulated, implemented, and tested independently by the student investigator using open-source tools

(Python 3.11, NCBI Entrez). No proprietary industrial software, institutional wet-lab facilities, or ghostwritten code was utilized.

XV. CONCLUSION

Generalized Pāṭha Codes demonstrate that ancient mnemonic recitation techniques from classical Sanskrit mathematics can solve a critical 21st-century challenge in molecular data storage. By arranging strand address bits into cyclic forward-reverse permutation passes anchored by deterministic pilots, GPC achieves 0.0% strand loss under 10-nucleotide translocation burst stalls with only 16.20% biological strand overhead. With verified combinatorial bounds ($B_E = 10K + 7$), an average-case $\mathcal{O}(M)$ decoding algorithm, an open-source testbed, and an interactive 1-click demonstration, GPC provides a practical, honest foundation for high-density archival DNA memory.

REFERENCES

1. G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, no. 6102, p. 1628, 2012.
2. N. Goldman et al., "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, pp. 77–80, 2013.
3. J. Bornholt et al., "A DNA-based archival storage system," *ACM ASPLOS*, pp. 637–649, 2016.
4. Oxford Nanopore Technologies, "R10.4.1 flow cell sequencing accuracy and translocation physics white paper," 2023.
5. R. R. Varshamov and G. M. Tenengolts, "Codes which correct single asymmetric errors," *Automatika i Telemekhanika*, vol. 26, no. 2, pp. 288–292, 1965.
6. C. Schoeny, A. Wachter-Zeh, R. Gabrys, and E. Yaakobi, "Codes for correcting a burst of deletions or insertions," *IEEE Trans. Inf. Theory*, vol. 63, no. 4, pp. 1971–1985, 2017.
7. B. van Nooten, "Binary numbers in Indian antiquity," *Journal of Indian Philosophy*, vol. 21, pp. 31–50, 1993.
8. R. Rajpopat, "In Pāṇini We Trust: Discovering the Algorithm for Rule Conflict Resolution in the Aṣṭādhyāyī," Ph.D. dissertation, University of Cambridge, 2022.
9. F. Sanger et al., "Nucleotide sequence of bacteriophage ΦX174 DNA," *Nature*, vol. 265, pp. 687–695, 1977.
10. C. Heckel et al., "Fundamental limits of DNA storage systems," *IEEE Trans. Inf. Theory*, vol. 65, no. 1, pp. 69–90, 2019.